

IMAGE PROCESSING · STUDY GUIDE

Chapter 3

Intensity Transformations & Spatial Filtering

A Complete Ground-Up Reference

From basic pixel-level intensity mappings to histogram equalization, spatial convolution, edge detection, and order-statistic filters. Includes worked examples, filter kernels, and practice questions.

Part 1 · Intensity Transformations

Part 2 · Bit-Plane Slicing

Part 3 · Histogram Equalization

Part 4 · Spatial Filtering Mechanics

Part 5 · Smoothing Filters

Part 6 · Sharpening & Laplacian

Part 7 · Gradient / Sobel

Part 8 · Non-Linear Filters

Part 9 · Practice Questions

Table of Contents

Part 1 · Intensity Transformation Functions	3
1.1 Image Negatives	3
1.2 Logarithmic Transformations	3
1.3 Power-Law (Gamma) Transformations	3
1.4 Linear Contrast Stretching	4
Worked Problem 1.1 — Contrast Stretching	4
Part 2 · Bit-Plane Slicing	5
Worked Problem 2.1 — Extraction & Reconstruction	5
Part 3 · Histogram Equalization	6
The Mathematical Algorithm (PMF, CDF)	6
Worked Problem 3.1 — Step-by-Step Equalization	7
Part 4 · Spatial Filtering Mechanics	8
4.1 Linear vs. Non-Linear Filters	8
4.2 Correlation vs. Convolution	8
Part 5 · Smoothing Spatial Filters (Low-Pass)	9
5.1 Box Filter	9
5.2 Weighted Average Filter	9
Part 6 · Sharpening Filters & The Laplacian	10
6.1 Mathematical Derivatives in 1D	10
6.2 The Laplacian (Second-Order)	10
6.3 Composite Sharpened Image	11
Worked Problem 6.1 — Laplacian & Composite Filtering	11
Part 7 · First-Order Edge Detection (Gradient / Sobel)	12
Worked Problem 7.1 — Sobel Response	12
Part 8 · Non-Linear Spatial Filters (Order-Statistics)	13
Worked Problem 8.1 — Median Filtering	13
Part 9 · Practice Questions	14

PART 1

Intensity Transformation Functions

Intensity transformations operate on individual pixels in the spatial domain. They map an input pixel intensity r to an output intensity s using a transformation function T . For an 8-bit grayscale image, both r and s fall within $[0, L-1]$ where $L = 256$.

$$s = T(r)$$

1.1 Image Negatives

Reverses the intensity levels, producing a photographic negative. Useful for enhancing white or grey detail embedded in dark regions.

$$s = (L - 1) - r$$

1.2 Logarithmic Transformations

Expands dark pixel values (low intensities) while compressing high intensities.

$$s = c \cdot \log(1 + r)$$

KEY NOTES

- **Use Case:** Enhancing images with concentrated low-intensity values — e.g., Fourier transform spectra where a few high values dominate.
- The **+1** inside the logarithm prevents $\log(0)$ when $r = 0$.
- The constant c scales the output to fit the maximum display limit (typically 255).

1.3 Power-Law (Gamma) Transformations

Curves the intensity mapping depending on the exponent γ (gamma):

$$s = c \cdot r^\gamma$$

$$\gamma < 1$$

Expands dark areas,
compresses bright areas.

Lightens the image.

$$\gamma = 1$$

Linear mapping.
No change to
image appearance.

$$\gamma > 1$$

Compresses dark areas,
expands bright areas.

Darkens the image.

USE CASES

- **Gamma Correction:** Correcting the non-linear display response of monitors.
- **Contrast Adjustment:** Selectively enhancing shadow or highlight detail.

1.4 Linear Contrast Stretching

Low-contrast images occur due to poor lighting or limited camera settings. Contrast stretching expands the existing intensity range $[r_{\min}, r_{\max}]$ to fill the complete dynamic range $[0, L-1]$:

$$s = (r - r_{\min}) \cdot L - 1 / (r_{\max} - r_{\min})$$

 WORKED PROBLEM 1.1 — CONTRAST STRETCHING

A low-contrast 3-bit image ($L = 2^3 = 8$, range $[0, 7]$) with matrix:

$$\begin{bmatrix} 2 & 2 & 3 \\ 2 & 4 & 3 \\ 3 & 3 & 4 \end{bmatrix}$$

F

1 Identify $r_{\min}$ and $r_{\max}$:

Lowest intensity: $r_{\min} = 2$ | Highest intensity: $r_{\max} = 4$

2 Apply the formula:

$$s = (r - 2) \cdot 7 / (4 - 2) = (r - 2) \cdot 3.5$$

3 Compute output values (rounded to nearest integer):

- $r = 2 \rightarrow s = (2-2) \times 3.5 = 0$
- $r = 3 \rightarrow s = (3-2) \times 3.5 = 3.5 \approx 4$
- $r = 4 \rightarrow s = (4-2) \times 3.5 = 7$

4 Contrast-stretched output matrix G :

$$\begin{bmatrix} 0 & 0 & 4 \\ 0 & 7 & 4 \\ 4 & 4 & 7 \end{bmatrix}$$

G

The original range $[2, 4]$ is now stretched to span the full $[0, 7]$ range.

PART 2

Bit-Plane Slicing

Each pixel value in an 8-bit image is composed of 8 binary bits. Separating these into individual planes reveals how much each bit contributes to the overall visual content.

$$\text{Pixel} = a_7 \cdot 2^7 + a_6 \cdot 2^6 + a_5 \cdot 2^5 + a_4 \cdot 2^4 + a_3 \cdot 2^3 + a_2 \cdot 2^2 + a_1 \cdot 2^1 + a_0 \cdot 2^0$$

MOST SIGNIFICANT BIT (MSB)

The **8th bit** (a_7). Contains the majority of visually significant structural information — coarse shapes and major edges.

LEAST SIGNIFICANT BIT (LSB)

The **1st bit** (a_0). Contains high-frequency details, subtle texture, and noise.

 WORKED PROBLEM 2.1 — BIT-PLANE EXTRACTION & RECONSTRUCTION

Given a 3-bit image ($L = 8$, values 0–7) with 2×2 matrix F :

$$\begin{bmatrix} 5 & 2 \\ 7 & 3 \end{bmatrix}$$

F

1 Convert each pixel to 3-bit binary:

Value	Binary (Bit3 · Bit2 · Bit1)
5	1 · 0 · 1
2	0 · 1 · 0
7	1 · 1 · 1
3	0 · 1 · 1

2 Extract individual bit planes:

BP_1 — LSB (Bit 1, weight = 2^0)

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

BP_2 — Middle Bit (weight = 2^1)

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

BP_3 — MSB (Bit 3, weight = 2^2)

$$\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

3 Reconstruct using only BP_1 and BP_3 :

$$\text{Reconstructed} = BP_1 \cdot 2^0 + BP_3 \cdot 2^2 = BP_1 \cdot 1 + BP_3 \cdot 4$$

$$\begin{bmatrix} 1 \cdot 1 + 1 \cdot 4 & 0 \cdot 1 + 0 \cdot 4 \\ 1 \cdot 1 + 1 \cdot 4 & 1 \cdot 1 + 0 \cdot 4 \end{bmatrix}$$

=

$$\begin{bmatrix} 5 & 0 \\ 5 & 1 \end{bmatrix}$$

Pixel (0,1) loses its middle bit contribution ($2 \rightarrow 0$), and pixel (1,1) loses bit 2 ($3 \rightarrow 1$).

PART 3

Histogram Equalization

A histogram plots the frequency of each intensity level in an image. Histogram equalization automatically redistributes intensity levels to approximate a **uniform distribution**, spreading contrast across the full intensity range.

The Mathematical Algorithm

For an image with total pixels N and discrete intensity levels r_k ($k = 0, 1, \dots, L-1$):

1 Compute the Probability Mass Function (PMF):

$$p_r(r_k) = n_k / N$$

where n_k is the number of pixels with intensity r_k .

2 Compute the Cumulative Distribution Function (CDF):

$$\text{CDF}(r_k) = \sum_{j=0 \text{ to } k} p_r(r_j)$$

3 Compute the equalized output intensity:

$$s_k = \text{round}[(L - 1) \cdot \sum_{j=0 \text{ to } k} p_r(r_j)]$$

 WORKED PROBLEM 3.1 — STEP-BY-STEP HISTOGRAM EQUALIZATION

Equalize a 3-bit image ($L = 8$, range $[0, 7]$) with $N = 64$ total pixels.

Pixel count per intensity:

$r_0=0$:
 $r_1=1$:
 $r_2=2$:
 $r_3=3$:
 $r_4=4$:
 $r_5=5$:
 $r_6=6$:

$r_7=7$:

r_k	n_k	PMF = $n_k/64$	CDF (cumulative)	$7 \times \text{CDF}$	Output s_k
0	8	0.125	0.125	0.875	1
1	10	≈ 0.156	0.281	1.967	2
2	12	0.188	0.469	3.283	3
3	7	≈ 0.109	0.578	4.046	4
4	15	≈ 0.234	0.812	5.684	6
5	5	≈ 0.078	0.890	6.230	6
6	4	≈ 0.063	0.953	6.671	7
7	3	≈ 0.047	1.000	7.000	7

Notice that intensities 4 and 5 both map to 6 (collision), and 6 and 7 both map to 7. This is a normal consequence of discrete equalization — perfect uniformity is only achievable for continuous distributions.

PART 4

Spatial Filtering Mechanics

Spatial filtering slides a small grid (a **mask**, **kernel**, or **filter**) over the image. At each position, a calculation on the pixel values under the mask generates a new value for the center pixel.

4.1 Linear vs. Non-Linear Filters

LINEAR FILTERS

Output is a **weighted sum** of input pixels:

$$g(x,y) = \sum_{s,t} w(s,t) \cdot f(x+s, y+t)$$

NON-LINEAR FILTERS

Output relies on **logical or sorting operations** on the local neighborhood — e.g., finding the median, minimum, or maximum value.

4.2 Correlation vs. Convolution

Correlation

The mask is placed directly over the image and slid across it **without modification**. Primarily used for **pattern matching / template matching** to locate a specific sub-image.

Convolution

The mask is **flipped 180°** (both horizontally and vertically) before being slid over the image. Forms the basis of linear system theory.

Original Mask:

A	B	C
D	E	F
G	H	I



Flipped 180° (Convolution-ready):

I	H	G
F	E	D
C	B	A

WHEN ARE THEY IDENTICAL?

When the spatial filter mask is **symmetric** (e.g., averaging mask, Gaussian, Laplacian). Flipping a symmetric mask returns the original mask — so Correlation = Convolution.

PART 5

Smoothing Spatial Filters (Low-Pass)

Smoothing filters blur an image to remove noise, eliminate small structures, and smooth transitions. They attenuate high-frequency transitions (sharp edges) while preserving low frequencies.

GOLDEN RULES OF SMOOTHING MASKS

1. All weights must be non-negative (≥ 0).
2. The sum of all weights must equal 1 — this prevents an overall brightness shift in the filtered image.

5.1 Box Filter (Simple Averaging)

All coefficients are equal — every pixel under the mask is treated with equal importance.

$$W_{\text{box}} = (1/9) \times$$

1	1	1
1	1	1
1	1	1

Sum of weights: $9 \times (1/9) = 1 \checkmark$

Drawback: Treats distant neighbours with equal weight as the center, causing more blurring of fine details compared to weighted alternatives.

5.2 Weighted Average Filter

Center pixel receives the highest weight; weights decrease with distance from center. Reduces blurring of sharp details compared to the box filter.

$$W_{\text{weighted}} = (1/16) \times$$

1	2	1
2	4	2
1	2	1

Sum of weights: $(1+2+1+2+4+2+1+2+1)/16 = 16/16 = 1 \checkmark$

Advantage: Approximates a Gaussian — centre-biased weighting reduces blurring while still smoothing noise.

PART 6

Sharpening Spatial Filters (High-Pass & Laplacian)

Sharpening highlights transitions, edges, and fine details while suppressing flat areas with uniform intensity.

GOLDEN RULES OF SHARPENING MASKS

1. The mask must contain both positive and negative weights.
2. The sum of all weights must equal 0 — flat uniform regions output 0, leaving only edges highlighted.

6.1 Mathematical Derivatives in 1D

FIRST DERIVATIVE (GRADIENT)

Measures the *rate of change*:

$$\partial f / \partial x = f(x+1) - f(x)$$

- Non-zero at **the onset** of an edge
- Used by Sobel, Prewitt operators

SECOND DERIVATIVE (LAPLACIAN)

Measures the *acceleration of change*:

$$\partial^2 f / \partial x^2 = f(x-1) + f(x+1) - 2f(x)$$

- Non-zero at **both sides** of an edge
- Zero at the edge midpoint (zero-crossing)

6.2 The Laplacian (Second-Order Isotropic Edge Detector)

The Laplacian is a second-order derivative operator that is **rotation-invariant** (isotropic) — its edge response does not depend on direction.

K_4 — 4-Connectivity

0	-1	0
-1	4	-1
0	-1	0

K_8 — 8-Connectivity

-1	-1	-1
-1	8	-1
-1	-1	-1

K_4 sum: $4 + 4 \times (-1) + 4 \times 0 = 0$ ✓

K_8 sum: $8 + 8 \times (-1) = 0$ ✓

K_8 detects diagonal edges too, making it more sensitive than K_4 but also noisier.

6.3 Composite Sharpened Image

The Laplacian alone yields only isolated edges on a black background. To sharpen while retaining original detail, we combine the Laplacian with the original:

$$g(x,y) = f(x,y) + L(x,y)$$

(if center of Laplacian mask is positive)

$$g(x,y) = f(x,y) - L(x,y)$$

(if center of Laplacian mask is negative)

WORKED PROBLEM 6.1 — LAPLACIAN & COMPOSITE FILTERING

Apply the 4-connectivity Laplacian (positive center = 4) to image F at center pixel $(1,1)$:

$$\begin{bmatrix} 3 & 1 & 4 \\ 1 & \mathbf{5} & 2 \\ 2 & 1 & 3 \end{bmatrix}$$

F (highlighted cell = center pixel $f(1,1) = 5$)

1 Compute Laplacian response $L(1,1)$:

$$\begin{aligned} L(1,1) &= 4 \cdot f(1,1) - f(0,1) - f(2,1) - f(1,0) - f(1,2) \\ &= 4(5) - 1 - 1 - 1 - 2 \\ &= 20 - 5 = \mathbf{15} \end{aligned}$$

2 Compute composite sharpened value $C(1,1)$:

Since the center weight is positive (+4), we add:

$$C(1,1) = f(1,1) + L(1,1) = 5 + 15 = \mathbf{20}$$

The center pixel is sharpened from 5 to 20, strongly emphasizing its contrast with neighbours.

PART 7

First-Order Edge Detection (Gradient Operators)

While the Laplacian uses second derivatives, gradient operators use **first derivatives** to find the steepness and direction of edges. They produce strong responses at transitions and weaker responses in flat regions.

7.1 Sobel Operators

The Sobel operator calculates spatial gradient approximations in the horizontal (G_x) and vertical (G_y) directions. It incorporates localized smoothing to reduce noise sensitivity.

H_x — Horizontal gradient

-1	0	1
-2	0	2
-1	0	1

H_y — Vertical gradient

-1	-2	-1
0	0	0
1	2	1

Gradient Magnitude:

$$G = \sqrt{(G_x^2 + G_y^2)}$$

Approximate (faster):

$$G \approx |G_x| + |G_y|$$

 WORKED PROBLEM 7.1 — SOBEL RESPONSE

Apply Sobel to this 3×3 neighbourhood:

$$\begin{bmatrix} 10 & 20 & 10 \\ 10 & 50 & 10 \\ 10 & 20 & 10 \end{bmatrix}$$

F

1 Compute G_x :

$$G_x = (-1 \cdot 10 + 1 \cdot 10) + (-2 \cdot 10 + 2 \cdot 10) + (-1 \cdot 10 + 1 \cdot 10) = 0 + 0 + 0 = 0$$

2 Compute G_y :

$$\begin{aligned} G_y &= (-1 \cdot 10 - 2 \cdot 20 - 1 \cdot 10) + 0 + (1 \cdot 10 + 2 \cdot 20 + 1 \cdot 10) \\ &= (-10 - 40 - 10) + (10 + 40 + 10) = -60 + 60 = 0 \end{aligned}$$

3 Analysis:

Both $G_x = 0$ and $G_y = 0$. The local region is **symmetric** about the center column ($G_x = 0$) and symmetric about the center row ($G_y = 0$). The high center value (50) surrounded by equal-sided values means there is no net directional edge. The Sobel operator reports no edge.

PART 8

Non-Linear Spatial Filters (Order-Statistics)

These filters do not use weighted sums. Instead, they **sort pixel values** within a neighbourhood and assign an output based on their rank. Each filter targets a specific type of noise.

MEDIAN FILTER

Selects the **middle value** after sorting.

Target: Salt-and-Pepper noise (random white/black pixels). Best at preserving sharp edges while removing both types.

MAX FILTER

Selects the **maximum value** in the neighbourhood.

Target: Pepper noise (black pixels, value = 0). Replaces dark noise pixels with the local brightest value.

MIN FILTER

Selects the **minimum value** in the neighbourhood.

Target: Salt noise (white pixels, value = 255). Replaces bright noise pixels with the local darkest value.

WORKED PROBLEM 8.1 — MEDIAN FILTERING

Apply a 3×3 median filter to the center pixel of this neighbourhood containing salt-and-pepper noise:

12	14	255
15	0	13
11	16	12

F (red = noisy pixels)

1 Extract all 9 values:

12	14	255	15	0	13	11	16	12
----	----	-----	----	---	----	----	----	----

2 Sort in ascending order:

0	11	12	12	13	14	15	16	255
---	----	----	----	----	----	----	----	-----

↑ Position: 1 2 3 4 **5 (median)** 6 7 8 9

3 Result: The median (5th of 9 values) = **13**.

The noisy center pixel (originally 0 — pepper noise) is replaced with **13**. The extreme values 0 and 255 are pushed to the ends of the sorted list and ignored.

PART 9

Practice Questions

Attempt each question fully before checking the hints.

QUESTION 1 — LOGARITHMIC TRANSFORMATIONS

An input image has intensities ranging from 0 to 255. Using $s = c \cdot \log(1 + r)$:

1. Calculate the value of c required to ensure the maximum output intensity is scaled to 255.
2. Find the output value s for an input intensity of $r = 127$.

HINT

1. At maximum $r = 255$: $255 = c \cdot \log(1 + 255) = c \cdot \log(256)$
 $\rightarrow c = 255 / \log(256) \approx \mathbf{45.98}$ (natural log) or $\approx \mathbf{105.9}$ (log base 10)
2. $s = c \cdot \log(1 + 127) = c \cdot \log(128)$. Substitute the value of c found above.

QUESTION 2 — HISTOGRAM EQUALIZATION

Given a 2-bit image ($L = 4$, values 0–3) with $N = 100$ total pixels:

$$n_0 = 40 \cdot n_1 = 30 \cdot n_2 = 20 \cdot n_3 = 10$$

Perform step-by-step histogram equalization to find the mapping from each r_k to s_k .

HINT

r_k	PMF	CDF	$3 \times \text{CDF}$	s_k
0	0.40	0.40	1.20	1
1	0.30	0.70	2.10	2
2	0.20	0.90	2.70	3
3	0.10	1.00	3.00	3

QUESTION 3 — 1D DERIVATIVES

A row of pixels has the following intensity values:

10	10	10	20	20	5	5
----	----	----	----	----	---	---

$f(x)$ at indices: 0 · 1 · 2 · 3 · 4 · 5 · 6

Calculate the **first derivative** $f(x+1)-f(x)$ and **second derivative** $f(x-1)+f(x+1)-2f(x)$ at each index.

HINT

First Derivative:

0	0	10	0	-15	0
---	---	----	---	-----	---

Second Derivative:

0	10	-10	-15	15	0
---	----	-----	-----	----	---

Note: The second derivative fires at *both edges* of each transition — it produces double responses at edges.

QUESTION 4 — COMPOSITE LAPLACIAN (NEGATIVE CENTER)

Given an image patch:

1	2	1
3	10	2
1	1	1

Using the 4-connectivity Laplacian with a **negative center**:

0	1	0
1	-4	1
0	1	0

K (negative-center variant)

1. Compute the Laplacian response $L(1,1)$ at the center.
2. Calculate the composite sharpened value $C(1,1) = f(1,1) - L(1,1)$.

HINT

$$1. L(1,1) = f(0,1) + f(2,1) + f(1,0) + f(1,2) - 4 \cdot f(1,1)$$

$$= 2 + 1 + 3 + 2 - 4(10) = 8 - 40 = -32$$

$$2. C(1,1) = f(1,1) - L(1,1) = 10 - (-32) = 42$$

When the center is negative, we subtract $L(1,1)$ to get the sharpened output — this is equivalent to adding the absolute Laplacian magnitude.